

Question No:10

DS	Session	Searching	Question Information	Level 1	Challenge 10
Question Description: An array A contains integers with the following constraints: 'A' contains elements in sorted order. Integer i occurs $i \cdot \text{floor}(\sqrt{i}) + \text{ceil}(i/2)$ times in the array. All elements are greater than or equal to 1. Fathima has given Q queries of type: L R: Find the number of distinct values present in subarray A[L...R]. Note: 1-based indexing is followed. Constraints: $1 \leq Q \leq 10^5$ $1 \leq L \leq R \leq 10^{13}$ Input Format: The first line contains an integer Q denoting the number of queries. Next Q lines contains two space-separated integers L, R, denoting the query. Output Format: Print the output in separate lines contains required number of distinct values. Example Sample Input: 2 1 3 1 6 Sample Output: 2					

Explanation

First few elements of the array A are 1, 1, 2, 2, 2, 3, 3, 3, 3, 3, ...

- For Query 1:-
 - Number of distinct elements in subarray $A[1...3]$ is 2.
- For Query 2:-
 - Number of distinct elements in subarray $A[1...6]$ is 3.

Logical Test Cases

Test Case 1

INPUT (STDIN)

2
3 3
2 6

EXPECTED OUTPUT

1
3

Test Case 2

INPUT (STDIN)

3
3 3
2 6
5 4

EXPECTED OUTPUT

1
3
1

Mandatory Test Cases

Test Case 1

KEYWORD

while (1<ans1)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

6

Test Case 2

TOKEN COUNT

400

Test Case 3

NLOC

56

Code:

```
1  #include <stdio.h>
2  #include <math.h>
3
4  long long countOccurrences(long long x)
5  {
6      long long root = (long long)sqrt((double)x);
7
8      while ((root + 1) * (root + 1) <= x)
9          root++;
10
11     while (root * root > x)
12         root--;
13
14     return x * root + (x + 1) / 2;
15 }
16
17 /* Returns total number of elements from 1 to x */
18 long long prefix(long long x)
19 {
20     if (x <= 0)
21         return 0;
22
23     long long root = (long long)sqrt((double)x);
24
25     while ((root + 1) * (root + 1) <= x)
26         root++;
27
28     while (root * root > x)
```

```

29         root--;
30
31     long long k = root - 1;
32
33     /*
34         Sum of:
35         i * floor(sqrt(i))
36
37         Complete blocks:
38         i = j^2 to (j+1)^2 - 1
39     */
40
41     long long sum2 = k * (k + 1) * (2 * k + 1) / 6;
42
43     long long temp = k * (k + 1) / 2;
44     long long sum3 = temp * temp;
45
46     long long sum4 =
47         k * (k + 1) * (2 * k + 1) *
48         (3 * k * k + 3 * k - 1) / 30;
49
50     long long complete =
51         2 * sum4 + 3 * sum3 + sum2;
52
53     /*
54         Partial block for root
55     */

```

```

56
57     long long start = root * root;
58
59     long long sumStart =
60         (start - 1) * start / 2;
61
62     long long sumX =
63         x * (x + 1) / 2;
64
65     long long partial =
66         root * (sumX - sumStart);
67
68     /*
69     Sum of ceil(i/2), i = 1 to x
70     */
71
72     long long half =
73         (x + 1) * (x + 1) / 4;
74
75     return complete + partial + half;
76 }
77
78
79 /* Find which integer is present at position pos */
80 long long findValue(long long pos)
81 {

```

```
82     long long low = 1;
83     long long high = 1;
84
85     while (prefix(high) < pos)
86         high *= 2;
87
88     while (low < high)
89     {
90         long long mid = low + (high - low) / 2;
91
92         if (prefix(mid) >= pos)
93             high = mid;
94         else
95             low = mid + 1;
96     }
97
98     return low;
99 }
100
101 int main()
102 {
103     int Q;
104     scanf("%d", &Q);
105
106     while (Q--)
107     {
```

```
108     long long L, R;
109
110     scanf("%lld %lld", &L, &R);
111
112     long long left = findValue(L);
113     long long right = findValue(R);
114
115     printf("%lld\n", right - left + 1);
116 }
117
118 return 0;
119 }
```

Output:

Test Case-1:

```
2
3 3
1
2 6
3
PS C:\Users\kesha\OneDrive\Documents\3sem\C>
```

Test Case-2:

```
PS C:\Users\kesha\OneDrive\Docum  
3  
3 3  
1  
2 6  
3  
5 4  
1  
PS C:\Users\kesha\OneDrive\Docum
```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236